Day 03 – Network Security Monitoring and Analysis

CyArt SOC Internship – Week 01

Intern Name: Harvinder Singh
Date: 20 March 2026

Executive Summary

This document covers the work I completed for Day 03 at CyArt, focused on network security monitoring and analysis. I used Wireshark to capture and inspect real network traffic on a Kali Linux virtual machine, Snort 3 as an Intrusion Detection System (IDS), and Scapy to generate controlled SYN-based test traffic.

The goal was to simulate a simple port scan against the local machine, see how that traffic looks in Wireshark, and then detect it using custom Snort rules. By the end of the lab, the Scapy traffic was captured in Wireshark and successfully triggered “SYN Packet Detected” and “Potential Port Scan” alerts in Snort, giving a clear end-to-end view of how reconnaissance activity can be monitored.

1. Introduction

In a real SOC or VAPT workflow, it is not enough to just run tools; we need to understand how packets move, how detection rules are written, and how alerts are generated. This task was designed with exactly that mindset.

For Day 03, I set up a small lab inside a Kali Linux VM and combined three tools:

- **Wireshark** to passively capture and analyze packets.
- **Snort 3** to act as an IDS watching the network interface.
- **Scapy** to craft and send custom packets that mimic a simple port scan.

The objectives were:

1. Capture different types of traffic (ICMP, DNS, TCP/HTTP, and SYN) using Wireshark and learn to filter them effectively.
2. Install and configure Snort 3 with a custom local rule file so that it can detect SYN packets and port scans on interface eth0.
3. Use a small Python script with Scapy to send SYN packets to multiple ports on the local host.
4. Confirm that this Scapy traffic is visible in Wireshark and that Snort generates alerts based on the rules I wrote.

The rest of the report walks through the environment, configuration steps, screenshots, and results in a clear and structured way.

2. Lab Environment

- **Operating System:** Kali Linux running inside Oracle VirtualBox
- **Network Mode:** NAT
- **Monitored Interface:** eth0
- **Interface IP:** 10.0.2.15

Tools used:

- **Wireshark** – packet capture and protocol analysis.
- **Snort++ 3.11.1.0 (Snort 3)** – network Intrusion Detection System (IDS) with Lua configuration.
- **Python 3 + Scapy** – for crafting and sending custom packets.

All testing was done inside the VM so nothing left the lab environment.

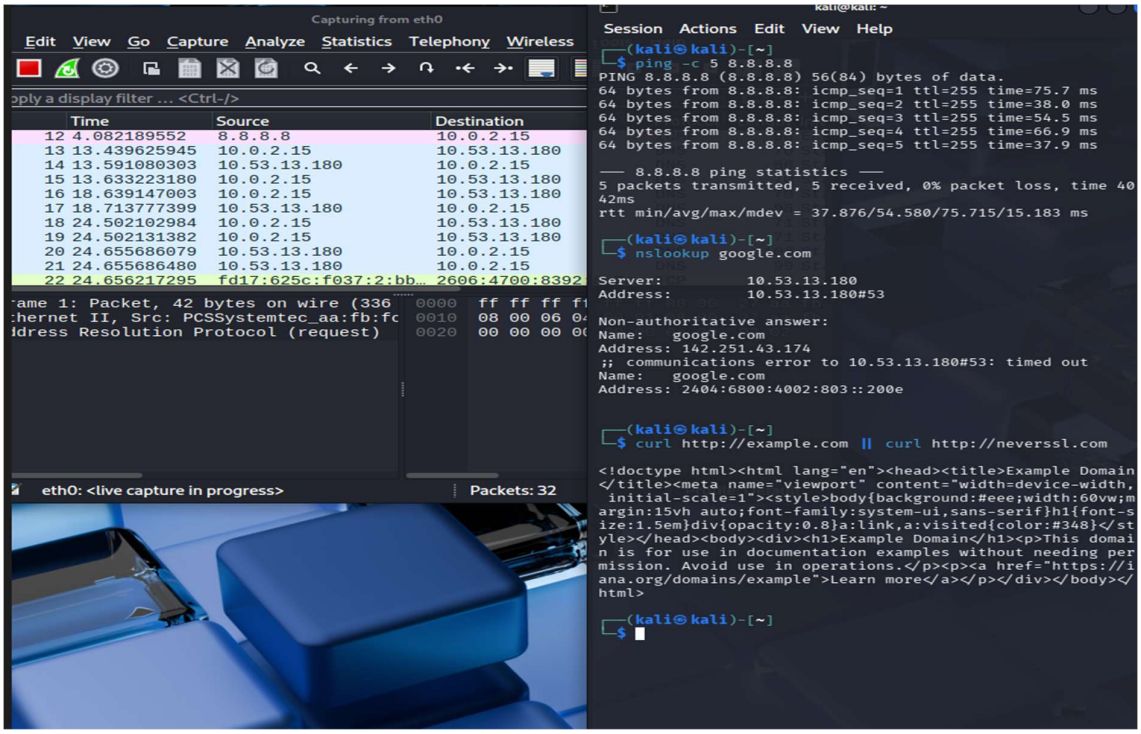
3. Wireshark Network Traffic Analysis

3.1 Capture Setup

I started Wireshark on Kali and selected the eth0 interface for capturing. To generate some "normal" traffic, I ran:

- ping -c 5 8.8.8.8
- nslookup google.com
- curl http://example.com (or curl http://neverssl.com)

Wireshark captured all of this traffic live. I then saved the capture as **capture.pcap** so it can be revisited later or attached along with the report.



0

Figure 1 – Wireshark capture and traffic generation commands
This figure shows Wireshark capturing traffic on interface eth0 while the terminal on the right runs ping, nslookup and curl to generate ICMP, DNS and HTTP traffic.

3.2 Protocol Filters

To better understand the captured traffic, I applied different Wireshark display filters for each protocol.

3.2.1 ICMP filter

Display filter:

Icmp

With this filter, Wireshark showed the echo request and echo reply packets generated by ping -c 5 8.8.8.8. The source address is 10.0.2.15 and the destination address is 8.8.8.8, which confirms basic connectivity from the Kali VM to the internet.

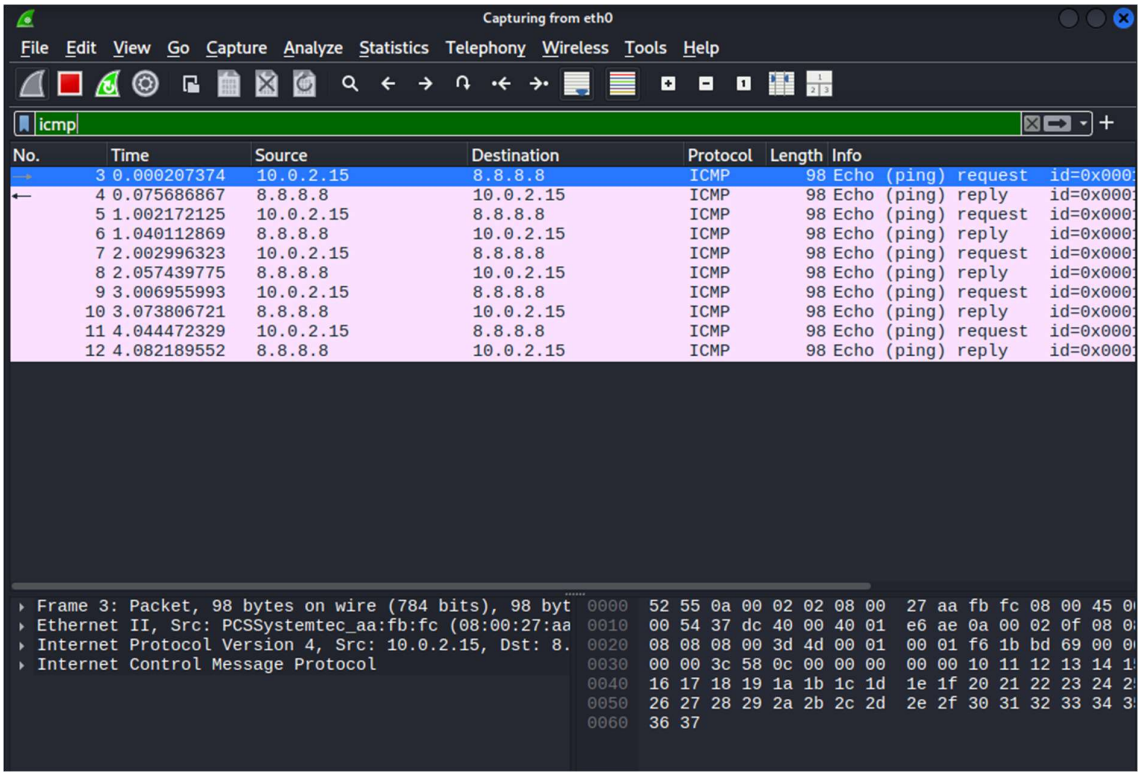


Figure 2 – ICMP packets filtered in Wireshark
Wireshark filtered with icmp shows echo requests and replies between 10.0.2.15 and 8.8.8.8, confirming basic network connectivity.

3.2.2 DNS filter

Display filter:

dns

Here I could see DNS queries from 10.0.2.15 to 10.53.13.180 resolving google.com to its IP addresses. This is a good way to visualize how name resolution actually looks on the wire.

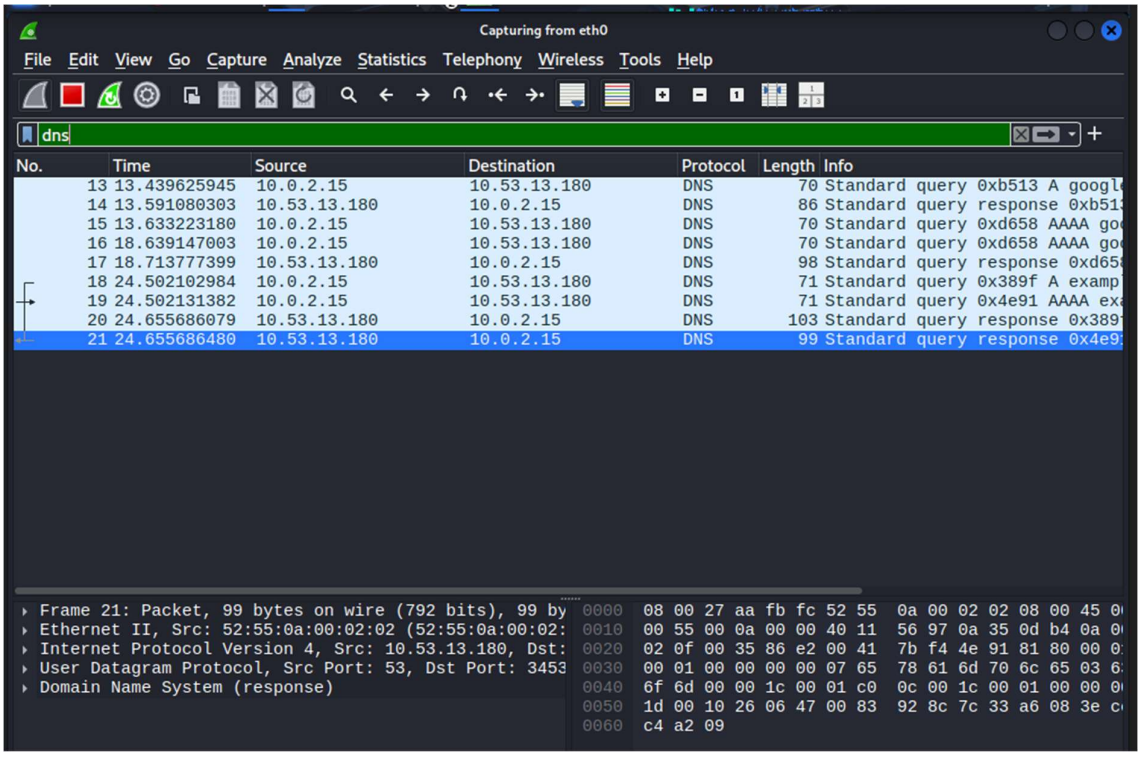


Figure 3 – DNS query and response for google.com
This figure shows Wireshark filtered with dns, displaying the query for google.com and the response from the DNS server 10.53.13.180.

3.2.3 TCP / HTTP filter

Display filter:

tcp

With the tcp filter, I focused on the TCP connections created by the curl command. Wireshark captured the three-way handshake (SYN, SYN/ACK, ACK) followed by the HTTP GET request and the 200 OK response from the web server. This makes it easy to see how application-layer HTTP traffic rides on top of TCP.

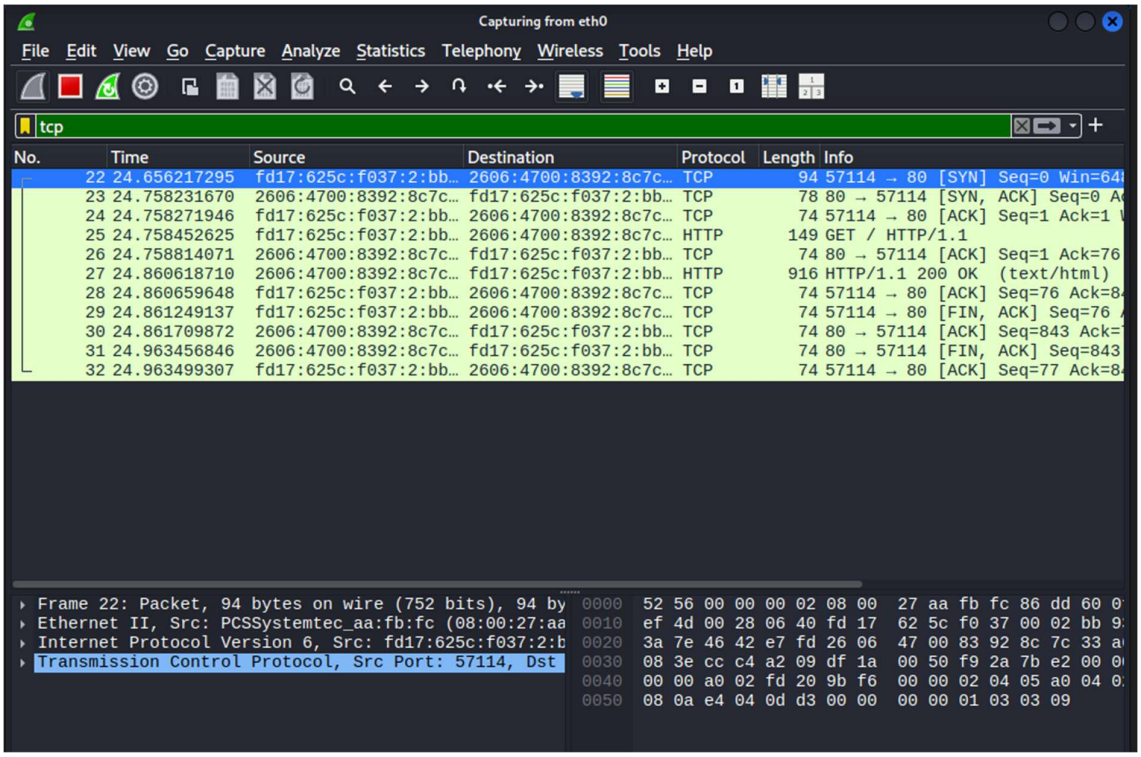


Figure 4 – TCP handshake and HTTP traffic
This figure shows the TCP three-way handshake and the HTTP GET/200 OK exchange generated by the curl command, filtered using tcp in Wireshark.

3.3 Suspicious SYN Pattern

After confirming normal ICMP, DNS and TCP traffic, I focused on packets that look like the start of a connection. These packets are important because many port-scanning tools rely on sending only SYN packets and not completing the full handshake.
For this, I used the following Wireshark display filter:

tcp.flags.syn==1 && tcp.flags.ack==0

This filter shows only those TCP packets where the SYN flag is set and the ACK flag is clear – in other words, initial SYN packets.
Once I ran my Scapy script (explained later in Section 5), Wireshark showed four such SYN packets:

- Source IP: 10.0.2.15
- Destination IP: 10.0.2.15
- Destination ports: **22, 80, 443 and 8080**

All four packets were clearly visible in the filtered view. This pattern matches a simple SYN scan where a host probes multiple service ports to see which ones respond.

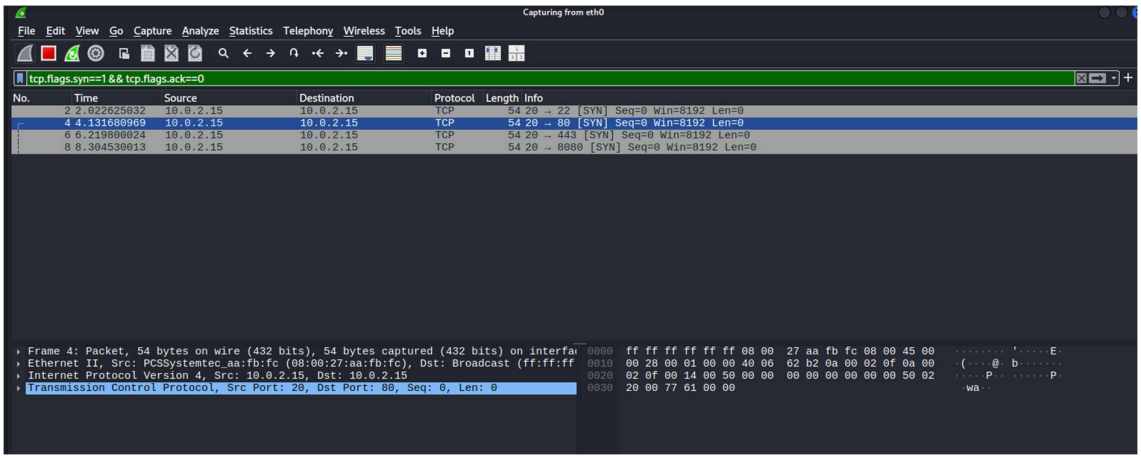


Figure 5 – Scapy-generated SYN packets in Wireshark
This figure shows Wireshark filtered with tcp.flags.syn==1 && tcp.flags.ack==0. Four SYN packets from 10.0.2.15 to ports 22, 80, 443 and 8080 on the same host are visible, representing a small port-scan pattern generated using Scapy.

4. Snort 3 IDS Configuration

4.1 Installation and Validation

On the same Kali VM, I installed Snort++ 3.11.1.0 (Snort 3). Unlike older Snort versions that use snort.conf, Snort 3 is configured with a Lua file:

- Main configuration file: /etc/snort/snort.lua

After installation, I validated the configuration to make sure there were no syntax errors and that the default rules loaded correctly:

```
sudo snort -c /etc/snort/snort.lua -T
```

The output showed that the configuration was successfully validated and that 221 rules were loaded (mostly default file and protocol inspection rules). This confirmed that Snort 3 was ready to run in IDS mode.

Figure X – Snort 3 configuration validation
This figure shows Snort validating /etc/snort/snort.lua successfully and reporting that 221 rules have been loaded with zero warnings.

```
ips policies rule stats
                id  loaded  shared enabled    file
                 0     221       0     221    /etc/snort/snort.lua
-------------------------------------------------------------
rule counts
       total rules loaded: 221
               text rules: 221
             option chains: 221
             chain headers: 3
-------------------------------------------------------------
port rule counts
            tcp     udp    icmp      ip
    any       2       0       0       0
  total       2       0       0       0
-------------------------------------------------------------
service rule counts          to-srv  to-cli
                  file_id:      219     219
                    total:      219     219
-------------------------------------------------------------
fast pattern groups
                to_server: 1
                to_client: 1
-------------------------------------------------------------
search engine (ac_bnfa)
appid: MaxRss diff: 2816
appid: patterns loaded: 300
-------------------------------------------------------------
pcap DAQ configured to passive.

Snort successfully validated the configuration (with 0 warnings).
o")~   Snort exiting
```

4.2 Creating Local Rules

To detect the SYN scan traffic generated with Scapy, I created a local rule file for CyArt:

- Local rule file path: /etc/snort/rules/local.rules

In this file, I added two simple but effective rules:

```
kali@kali: ~
Session  Actions  Edit  View  Help
  GNU nano 8.7.1                                   /etc/snort/rules/local.rules *
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
# ----------------
# LOCAL RULES
# ----------------
# This file intentionally does not come with signatures.  Put your local
# additions here.
alert tcp any any -> any any (msg:"SYN Packet Detected"; flags:S,12; sid:1000001; rev:1;)
alert tcp any any -> any 1:1024 (msg:"Potential Port Scan"; flags:S,12; sid:1000002; rev:1;)
```

Explanation:

- **SYN Packet Detected**
  This rule fires on any TCP packet that has the SYN flag set. It gives basic visibility into connection attempts.
- **Potential Port Scan**
  This rule fires when a SYN packet targets any destination port between 1 and 1024. Because many well-known services use ports in this range, a series of SYN packets here often indicates port-scanning behaviour.

These two rules are enough to detect the Scapy traffic used later in the lab.

4.3 Including local.rules in snort.lua

By default, Snort 3 does not automatically use local.rules. I had to explicitly include it inside the ips configuration block in /etc/snort/snort.lua:

```
ips =
{
    -- use this to enable decoder and inspector alerts
    --enable_builtin_rules = true,

    -- use include for rules files; be sure to set your path
    -- note that rules files can include other rules files
    -- (see also related path vars at the top of snort_defaults.lua)

    variables = default_variables
rules = [[
        include /etc/snort/rules/local.rules
    ]]

}
```

This tells Snort to:

- Use default_variables (such as HOME_NET, EXTERNAL_NET, etc.), and
- Load all rules defined in /etc/snort/rules/local.rules.

After making this change, I ran the validation command again:

```
sudo snort -c /etc/snort/snort.lua -T
```

The configuration was still valid and the rule statistics showed that the custom rules were now included.

4.4 Running Snort in IDS Mode on eth0

With the configuration and rules in place, I started Snort in IDS mode on the eth0 interface:
bash

```
sudo snort -c /etc/snort/snort.lua -i eth0 -A alert_fast
```

Options used:

- -c /etc/snort/snort.lua – tells Snort which configuration file to use.
- -i eth0 – selects the network interface to monitor.

- -A alert_fast – prints alerts in a concise console format.

Snort printed “Commencing packet processing” and showed that it was listening on eth0, indicating that it was actively inspecting live traffic and ready to generate alerts when the Scapy scan was executed.

```
search engine (ac_bnfa)
                instances: 2
                 patterns: 438
            pattern chars: 2602
               num states: 1832
         num match states: 392
             memory scale: KB
             total memory: 71.2812
           pattern memory: 19.6484
        match list memory: 28.4375
        transition memory: 22.9453
appid: MaxRss diff: 3200
appid: patterns loaded: 300
-------------------------------------------------
pcap DAQ configured to passive.
Commencing packet processing
Retry queue interval is: 200 ms
++ [0] eth0
```

5. Scapy Packet Generation

5.1 Purpose

Once Wireshark and Snort were ready, the next step was to generate controlled test traffic. Instead of using nmap, I used **Scapy** so that every field of the packet is under my control. The idea was to simulate a small SYN-based port scan against the local Kali machine and see how it appears in Wireshark and in Snort alerts.

5.2 Test Script (test_packets.py)

I created a Python script named **test_packets.py** with the following content:

```python
from scapy.all import IP, TCP, send

target_ip = "10.0.2.15"   # tumhara eth0 IP
ports = [22, 80, 443, 8080]

for p in ports:
    pkt = IP(dst=target_ip) / TCP(dport=p, flags="S")
    print(f"Sending SYN to {target_ip}:{p}")
    send(pkt, verbose=0)
S
```

Explanation:

- target_ip is set to the IP address of eth0 (10.0.2.15), so all packets stay inside the lab.
- The ports list contains the four target ports for the scan: SSH (22), HTTP (80), HTTPS (443) and a common web port (8080).
- For each port, Scapy builds an IP/TCP packet with only the SYN flag set and sends it once.

5.3 Execution

With Snort already running in IDS mode on eth0, I executed the script:

sudo python3 test_packets.py

The terminal output confirmed that packets were being sent:

- Sending SYN to 10.0.2.15:22
- Sending SYN to 10.0.2.15:80
- Sending SYN to 10.0.2.15:443
- Sending SYN to 10.0.2.15:8080

Scapy printed warnings such as “MAC address to reach destination not found. Using broadcast.” This is normal in a virtualized NAT setup and simply means Scapy used broadcast at the Ethernet layer; the packets were still delivered successfully.

Wireshark's SYN-only filter (tcp.flags.syn==1 && tcp.flags.ack==0) clearly showed one SYN packet to each of the four ports, confirming that the script behaved as expected.

Figure Z – Scapy script sending SYN packets
This figure shows the test_packets.py script running in the terminal and sending SYN packets to 10.0.2.15 on ports 22, 80, 443 and 8080.

```
└─$ sudo python3 test_packets.py
[sudo] password for kali:
Sending SYN to 10.0.2.15:22
WARNING: MAC address to reach destination not found. Using broadcast.
Sending SYN to 10.0.2.15:80
WARNING: MAC address to reach destination not found. Using broadcast.
Sending SYN to 10.0.2.15:443
WARNING: more MAC address to reach destination not found. Using broadcast.
Sending SYN to 10.0.2.15:8080
WARNING: MAC address to reach destination not found. Using broadcast.
```

6. Snort Alert Results

With Snort monitoring eth0 and the Scapy SYN packets on the wire, the custom CyArt rules began to fire.

As soon as the script ran, Snort printed a series of alerts similar to:

- [1:1000002:1] "Potential Port Scan" {TCP} 10.0.2.15:20 -> 10.0.2.15:22
- [1:1000001:1] "SYN Packet Detected" {TCP} 10.0.2.15:20 -> 10.0.2.15:22
- [1:1000002:1] "Potential Port Scan" {TCP} 10.0.2.15:20 -> 10.0.2.15:80
- [1:1000001:1] "SYN Packet Detected" {TCP} 10.0.2.15:20 -> 10.0.2.15:80
- [1:1000002:1] "Potential Port Scan" {TCP} 10.0.2.15:20 -> 10.0.2.15:443
- [1:1000001:1] "SYN Packet Detected" {TCP} 10.0.2.15:20 -> 10.0.2.15:443
- [1:1000001:1] "SYN Packet Detected" {TCP} 10.0.2.15:20 -> 10.0.2.15:8080

Snort's statistics at the end of the run showed:

- **Packets analyzed:** 13
- **Total alerts:** 7
- **Alerts logged:** 7

This matches the logic of the two rules:

- For ports **22**, **80** and **443** (which are inside the 1–1024 range), both rules triggered:
  - SYN Packet Detected (because of the SYN flag),
  - Potential Port Scan (because the destination port is between 1 and 1024).
- For port **8080**, only the SYN Packet Detected rule triggered, because 8080 is outside the configured port-scan range.

Figure W – Snort alerts for SYN and port scan rules

This figure shows Snort 3 generating “SYN Packet Detected” and “Potential Port Scan” alerts for Scapy traffic to ports 22, 80, 443 and 8080, along with a statistics summary indicating that 7 alerts were raised and logged.

```
03/20-06:58:48.984158 [**] [1:1000002:1] "Potential Port Scan" [**] [Priority: 0] {TCP} 10.0.2.15:20 -> 10.0.2.15:22
03/20-06:58:48.984158 [**] [1:1000001:1] "SYN Packet Detected" [**] [Priority: 0] {TCP} 10.0.2.15:20 -> 10.0.2.15:22
03/20-06:58:51.089064 [**] [1:1000002:1] "Potential Port Scan" [**] [Priority: 0] {TCP} 10.0.2.15:20 -> 10.0.2.15:80
03/20-06:58:51.089064 [**] [1:1000001:1] "SYN Packet Detected" [**] [Priority: 0] {TCP} 10.0.2.15:20 -> 10.0.2.15:80
03/20-06:58:53.168047 [**] [1:1000002:1] "Potential Port Scan" [**] [Priority: 0] {TCP} 10.0.2.15:20 -> 10.0.2.15:443
03/20-06:58:53.168047 [**] [1:1000001:1] "SYN Packet Detected" [**] [Priority: 0] {TCP} 10.0.2.15:20 -> 10.0.2.15:443
03/20-06:58:55.247858 [**] [1:1000001:1] "SYN Packet Detected" [**] [Priority: 0] {TCP} 10.0.2.15:20 -> 10.0.2.15:8080
-- [0] eth0
--------------------------------------------------
Packet Statistics
--------------------------------------------------
daq
        received: 9
        analyzed: 9
           allow: 9
        rx_bytes: 444
--------------------------------------------------
codec
           total: 9          (100.000%)
             arp: 4          ( 44.444%)
             eth: 9          (100.000%)
            ipv4: 5          ( 55.556%)
             tcp: 5          ( 55.556%)
--------------------------------------------------
Module Statistics
--------------------------------------------------
appid
         packets: 5
  ignored_packets: 5
--------------------------------------------------
arp_spoof
         packets: 4
--------------------------------------------------
binder
     raw_packets: 9
        inspects: 9
--------------------------------------------------
detection
        analyzed: 9
      hard_evals: 5
          alerts: 7
    total_alerts: 7
          logged: 7
--------------------------------------------------
ips_actions
           alert: 7
--------------------------------------------------
```

7. Key Learnings and Challenges

- **Wireshark filters and visualisation**
  I became more comfortable using Wireshark display filters such as icmp, dns, tcp and tcp.flags.syn==1 && tcp.flags.ack==0. This helped me quickly move from a noisy capture to exactly the packets I needed (for example, the Scapy SYN packets).
- **Snort 3 configuration with Lua**
  I learned that Snort 3 uses a Lua-based configuration file (snort.lua) instead of the classic snort.conf. Including a custom rule file inside the ips block and validating the configuration with snort -T was a new but important step.
- **Writing and testing IDS rules**
  Creating two simple rules (“SYN Packet Detected” and “Potential Port Scan”) showed

how detection logic maps to real traffic. Watching those rules fire on the Scapy packets made the rule behaviour very clear.

- **Using Scapy for controlled test traffic**
  Instead of relying on external tools, I used Scapy to craft specific SYN packets to the ports I wanted. This gave full control over the test scenario and is a useful technique for both VAPT and SOC rule testing.
- **Interpreting Snort statistics**
  I also learned how to read Snort's module statistics. For example, during the test run the detection engine analysed 13 packets and generated 7 alerts, while the codec module reported 9 fully decoded packets (5 IPv4 and 4 ARP). This helped me understand that different modules track different counters, so the numbers do not always match exactly but still make sense together.

8. Conclusion

This Day 03 task for CyArt successfully tied together packet capture, IDS configuration, and traffic generation into a single, coherent workflow.

- Wireshark showed how different protocols look on the wire and made it easy to identify SYN-only traffic.
- Snort 3, configured with a small set of custom rules, was able to detect SYN packets and potential port scans aimed at the local host.
- Scapy generated exactly the kind of traffic needed to exercise those rules, and the entire flow from packet generation → capture → detection was clearly visible.

This small lab is a good foundation for more advanced work, such as tuning rules, adding thresholds for scan detection, or integrating Snort alerts into a central monitoring system. It also serves as solid documentation to show the approach and results to mentors and future teammates at CyArt.